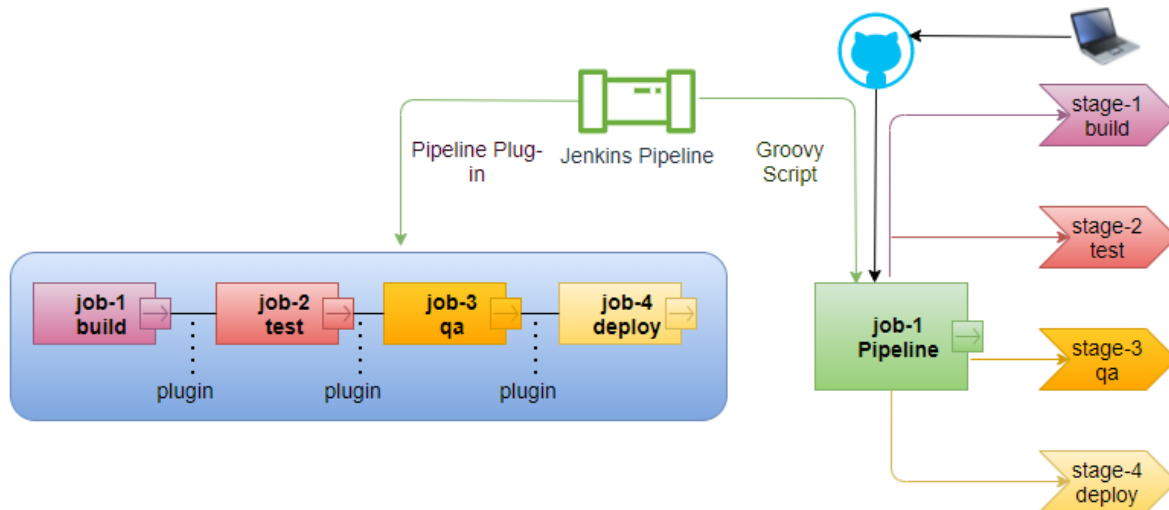


JenkinsFiles? It is a text file that contains the definition of a Jenkins pipeline, and it is checked into the source control repository (SCR). Pipeline as a code.

How to Integrate the Jobs?



Pipeline as a code

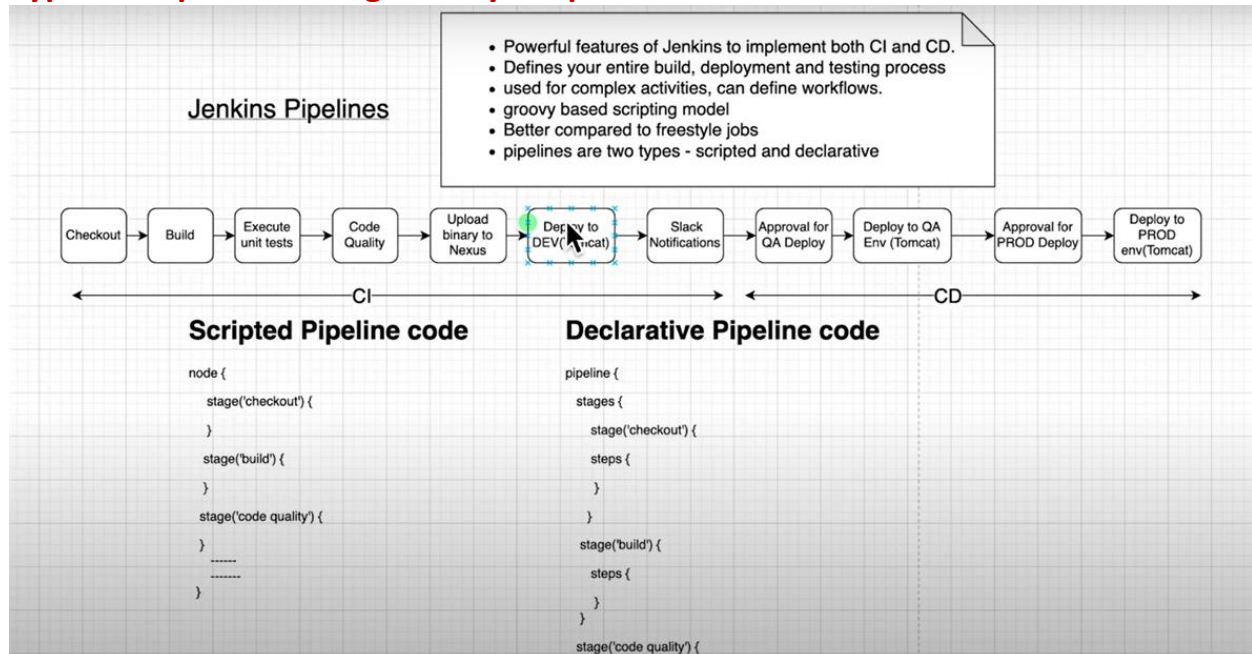
Limitations of Pipeline Using Plugins

- Using Jenkins UI Interface
- All jobs are executed sequentially, can't skip any Job from the middle.
- If job4 fails, means our tasks are not completed, our tasks will be completed only **once complete pipeline** will be executed, then we have to correct the job4 and execute the pipeline again, second time pipeline will be executed from the starting (Time consuming).
- No concept of looping/conditions.
- All jobs are executed only in master till master is live.

Benefits of Pipeline Using Groovy Scripts

- Using Jenkins UI Interface, as well as using Jenkins files (pipeline as a code).
- We can skip Job3 from the middle while Job 3 is still there (condition).
- In case of Job4 failure, second time we can only restart Stage4.
- Groovy Scripts have In-build suite of plugins.
- We can apply conditions and looping in Groovy Scripts.
- We can distribute the load with agents.

Types of Pipelines Using Groovy Scripts?



- Whereas the scripted pipeline is a traditional way of writing the code. In this pipeline the Jenkinsfile is written on the Jenkins UI Instance.
- In Declarative Pipeline Code Validation happens at startup.

Script

```
1 node {
2     stage("Build") {
3         echo "Building"
4     }
5
6     stage("Test") {
7         echo 123
8     }
9 }
```

In Case of scripted pipeline Build Stage will executed successfully, will throw an exception at the time of Test stage execution, will in the case of Declarative pipeline script will throw exception at startup before execution of the Build stage.

- In Declarative pipeline we can restart the specific stage in case of failure.
- In Declarative Pipeline have option of skip the stage using the conditions.

```

stage("Test") {
    when {
        beforeAgent true
        expression {
            env.SOMETHING == "lorem ipsum"
        }
    }
}

```

- Declarative Pipeline Options?

```

pipeline {
    agent any
    options {
        timestamps()
        ansiColor("xterm")
    }

    stages {
        stage("Build") {
            options {
                timeout(time: 1, unit: "SECOND")
            }
            steps {
                sh 'printf "\\e[31mHello World\\e[0m\\n"'
                sh "sleep 5s"
            }
        }
    }
}

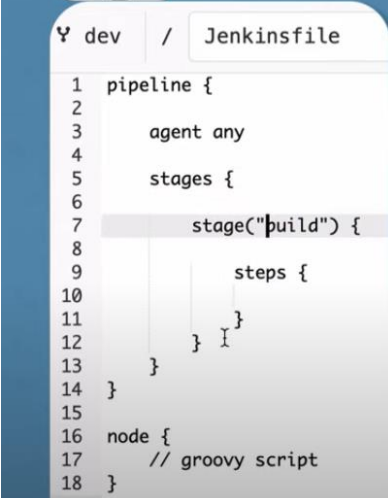
```

Global options at pipeline level means it can be applied at any stages.

Declarative Pipeline?

Syntax:

Required fields of Jenkinsfile



```

1 pipeline {
2
3     agent any
4
5     stages {
6
7         stage("build") {
8
9             steps {
10
11             }
12         }
13     }
14 }
15
16 node {
17     // groovy script
18 }

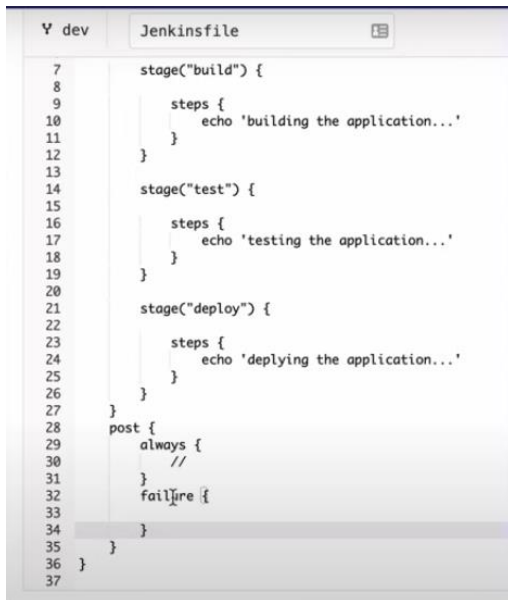
```

- "pipeline" must be top-level
- "agent" - where to execute
- "stages" - where the "work" happens
- "stage" and "steps"

Benefits of Pipeline (Declarative) Using Groovy Scripts

- Using Jenkins UI Interface, as well as using Jenkins files (pipeline as a code).
- We can skip Job3 from the middle while Job 3 is still there (condition).
- In case of Job4 failure, second time we can only restart Stage4.
- Groovy Scripts have In-build suite of plugins.
- We can apply conditions and looping in Groovy Scripts.
- We can distribute the different stage with different agents.
- Validation happens at startup.
- Options

Post Attribute in Jenkins



```
7 stage("build") {
8
9     steps {
10         echo 'building the application...'
11     }
12 }
13
14 stage("test") {
15
16     steps {
17         echo 'testing the application...'
18     }
19 }
20
21 stage("deploy") {
22
23     steps {
24         echo 'deploying the application...'
25     }
26 }
27
28 post {
29     always {
30         //
31     }
32     failure {
33
34     }
35 }
36
37 }
```

Execute some logic
AFTER all stages executed

Conditions:

- always
- success
- failure

Define Conditionals for each stage, using the expression within when block
UC: execute the "test" stage only on dev branch

```
stage("test") {
    when {
        expression {
            BRANCH_NAME == 'dev' || BRANCH_NAME == 'master'
        }
    }
    steps {
        echo 'testing the application...'
    }
}
```

How get the branch name?

Environmental Variable provided by Jenkins

What variables are available from Jenkins? [#localhost:8080/env-vars.html](#)

```

//CODE_CHANGES = getGitChanges()
def gv
pipeline {
    agent any

    tools{ // build tool
        maven 'mymaven'
    }// tool

    environment { //available to all stage
        NEW_VERSION='1.3.0'
        SERVER_CREDENTIALS = credentials('1426dee9-d21f-4e79-bce2-50f2905927f6')
    }// environment

    parameters { // parameters pass suitable candidates for expressions

        // string(name, defultValue, description)
        //choice(name,choices,description)
        //booleanParam(name, defaultvalue,description)
        string(name: 'SELECT_VERSION', defaultValue: '1.1.0', description: 'string
version parameter to be deployed')
        choice(name: 'CHOICE_VERSION',choices: ['1.1.0','1.3.0','1.5.0'],description:
'choice parameter')
        booleanParam(name:'executeTests',defaultValue: 'true',description:
'boolean parameter')

    }// parameters

    stages {
        stage('init'){
            steps{
                script{
                    gv=load "script.groovy"
                }
            }
        }
    }
}

```

```
}// init-stage
stage('1-build') {
    steps {
        echo 'Hello Build'
        echo "custome variable double ${NEW_VERSION}"
        echo "server credentials ${SERVER_CREDENTIALS}"
    }
}

stage('2-test') {

    when{
        expression { // inbuild environment
            env.BRANCH_NAME == 'master' || env.BRANCH_NAME == 'main'
        }
    }

    steps {
        echo 'Hello Test'
    }
}

stage('3-deploy') {
    when{
        expression {
            params.executeTests

        }

    }

    steps {
        echo 'Hello Deploy'
        echo "deploying with choice version ${params.CHOICE_VERSION}"
        echo "deploying with string version ${params.SELECT_VERSION}"
    }
}
```

```

stage('5-execute groovyscript'){
    steps{
        script {
            // to execute groovy script define globally def gv
            // gv.execute()
        }
        // echo 'execute groovy scripts'
    }
} //5- execute groovy

} //stages

post{
    // build status or status changes

    always{
        echo ' send email always'
    } //always

    success{
        echo ' send email on success'
    } //always

} //post-stages after all stages are done
} //Pipeline

```

Using external Groovy Script

Clean the Jenkinsfile and move the logic in external groovy script files, add the functionality using the script tag.

step1: write a groovy script (script.groovy)

```

1  def function() {
2      echo 'building the application...'
3  }
4
5  return this

```

step2: Define the global variable to hold the groovy script

```

1  def gv
2
3  pipeline {
4      agent any
5      parameters {
6          choice(name: 'VERSION', choices: ['1.1.0', '1.2.0', '1.3.0'], description: '')
7          booleanParam(name: 'executeTests', defaultValue: true, description: '')
8      }

```

step3: In init stage load the groovy script

```

7      stages {
8          stage("init") {
9              steps {
10                 script {
11                     gv = load "script.groovy"
12                 }
13             }
14         }

```

step4: use function written in groovy script in jenkinsfile

step4.1: Update the groovy script

```

1  def buildApp() {
2      echo 'building the application...'
3  }
4
5  def testApp() {
6      echo 'testing the application...'
7  }
8
9  def deployApp() {
10     echo 'deplying the application...'
11     echo "deploying version ${params.VERSION}"
12 }
13
14 return this

```

step4.2: Use functions of groovy script in jenkinsfile


```

24     stage("test") {
25         when {
26             expression {
27                 params.executeTests
28             }
29         }
30         steps {
31             script {
32                 gv.testApp()
33             }
34         }
35     }

```

All the environmental variables In Jenkinsfile are available in groovy script.

Delegate the job in specific agent:

```

pipeline{
    agent {
        label 'label of agent'
    } // agent
} // pipeline

```

Execute the job in parallel:

A parallel keyword encounters Jenkins ensure that stage inside parallel must be executed parallel.

```

pipeline{
  agent none
  stages {

    stage ('Non Parallel Stage' {
      agent {
        label: 'master'
      }
      step{
        echo "This stage will be executed First"
      }
    } // stage non [arallel

    stage ('Run test'){
      parallel{
        stage("Test on Windows") {
          agent {
            label: 'window-node'
          }
          step{
            echo "This stage will be executed on window-node"
          }
        } // stage window

        stage("Test on Linux") {
          agent {
            label: 'linux-node'
          }
          step{
            echo "This stage will be executed on linux-node"
          }
        } // stage linux
      } // parallel
    } // stage run
  } // pipeline
}

```

General
Build Triggers
Advanced Project Options
Pipeline

Pipeline

Definition

Pipeline script
Pipeline script
Pipeline script from SCM

3
4
5
6
7
8
9
10
11
12

```

stages {
  stage('Hello') {
    steps {
      echo 'Hello World'
    }
  }
}

```

☒ Use Groovy Sandbox

Pipeline Syntax

multibranch pipeline:

solve the issue to build for each branch

plugin- Pipeline: Multibranch

project type: Multibranch Pipeline

automatically find the available branches in the repository, start build pipeline against each pipeline

multiconfiguration pipeline:

suitable for projects that needs a large number of different configurations, such as testing on multiple environments, platform-specific tasks etc.

plugin- Pipeline: Multiconfiguration

project type: Multiconfiguration Pipeline